

PROYECTO DE ESTRUCTURAS DE DATOS
(PROYECTO EN GRUPOS DE 3, SUSTENTACIÓN EN GRUPOS, NOTA INDIVIDUAL)

CONSIDERACIONES GENERALES

- El proyecto está disponible (20 días) hasta el lunes 25 de mayo a las 8:00 AM.
- En BrightSpace debe ser subido el link de GitHub donde se realizó el proyecto de forma colaborativa por el equipo.
- La sustentación del proyecto es en grupo, allí se validará la propiedad intelectual del trabajo.
- La sustentación es un factor de 0 a 1 que se multiplica por el valor obtenido en la nota del proyecto entregado. Ej., si la nota trabajo es 5 y la sustentación es 0.8, la nota definitiva del parcial sería 4.0
- El fraude puede ocasionar la apertura de un proceso disciplinario.
- Tiempo que estimo que usted debería tardar haciendo el proyecto (20 horas)

El Caso del Detective

Para el proyecto final ustedes desarrollarán un juego que simula una investigación criminal en una ciudad, donde el usuario asumirá el rol de un detective que deberá recolectar 10 pistas ocultas para reunir suficiente evidencia y acusar al sospechoso correcto entre los registrados en el caso. Más específicamente, la ciudad es una cuadrícula que tiene un área de juego de 9 filas por 9 columnas (81 nodos), donde cada nodo representa una ubicación (callejón, esquina, edificio, café, archivo, hospital, etc.). El tablero estará completamente rodeado por "edificios" que el detective (a quien le solicitaremos datos básicos) no podrá atravesar porque se saldría de la cuadrícula.

Es importante aclarar que para el juego ustedes no deberán implementar nada relacionado con librerías de interfaz gráfica, sólo deberán imprimir en modo texto el tablero, las jugadas y la información al usuario para que él pueda observar su avance y tenga una buena experiencia de juego (busquen ser creativos y estéticos para lograrlo). Para el juego se podrán usar los siguientes TADs (pilas, colas, árboles, listas enlazadas, colas de prioridad, Tablas Hash) no es posible usar arreglos ni matrices, si para imprimir (presentar las cosas al usuario) desea usar arreglos lo puede hacer, pero no para la jugabilidad.

Para la realización del juego se espera que haga uso de la Programación Orientada a Objetos (POO), es decir, que al crear entidades dentro del juego (Detective, Pista, Sospechoso, Testigo, Ubicación, etc.), ellas deberán ser creadas con clases y respetando los principios de POO.

Por favor tengan en cuenta las siguientes características para realizar el juego, léanlas con atención y si queda alguna duda el profe será su cliente y podrá consultarle lo que necesite.

1. La solución al juego debe estar desarrollada en el lenguaje de programación C++, no puede estar en otro lenguaje, C, Python, etc. No serán lenguajes permitidos.
2. Puede utilizar la librería STL para hacer uso de las estructuras de datos que necesite.

3. El mapa de la ciudad es una lista múltiplemente enlazada para poder desplazarse por ella hacia arriba, abajo, izquierda o derecha.
4. Cada ubicación de la ciudad será un nodo de la lista, constrúyalo con los datos que considere adecuados (coordenada, contenido, estado de descubierto/oculto, etc.).
5. Los laterales del tablero serán edificios que el detective no podrá atravesar, ellos quedarán marcados con el signo “#”.
6. El Detective en el tablero estará marcado con la letra “D” y su aparición en el tablero al comenzar la partida será de manera aleatoria, es decir, que puede aparecer en cualquier parte, excepto donde haya Pistas, Testigos o Callejones cerrados (estos serán explicados más adelante).
7. Las Pistas (que están ocultas para el usuario) pueden ser de 4 tipos (Huella "H", Coartada "C", Testimonio "T", Prueba forense "P"). En el tablero deberán aparecer de manera aleatoria 10 de ellas (no importa la combinación), y el juego avanzará a la fase de acusación cuando el detective haya recogido las 10 pistas ocultas. En este punto, se le pedirá al usuario que acuse a uno de los sospechosos.
8. Cada que el detective haya recogido una Pista, el sistema le avisará al usuario que ha recogido una pista y qué tipo es. Adicionalmente, esa pista deberá ir a una estructura de datos (Pila), tengan en cuenta que el usuario podrá utilizar las pistas para generar acciones en el juego, pero sólo podrá utilizar la última pista hallada, y luego de ella la penúltima, y así hasta llegar a la primera pista recogida.
9. El juego tendrá un total de 16 Callejones cerrados que serán marcados con el símbolo “[”. El detective no podrá atravesarlos, aunque el sistema le avisará cuando esté intentando moverse hacia uno. Por ejemplo, si el usuario desea moverse a la derecha y allí hay un callejón cerrado, este se hará visible (mostrarse en el tablero en pantalla) pero el detective no cambiará su posición.
10. Tengan en cuenta que para saber hacia dónde se moverá el detective “D”, el usuario le dará una letra en mayúscula que lo indica. Así (W es arriba, S es abajo, A es izquierda y D es derecha). Siempre que el usuario indique una instrucción para mover al detective, se debe mostrar el tablero con las nuevas condiciones logradas. Nota: como la letra D se usa para el personaje, se sugiere no confundir; si lo prefieren pueden marcar al detective con otra letra como “I” (Investigador) y dejar D para el movimiento.
11. Puntaje. Para hacer competitivo el juego es necesario que le demos un puntaje, y ello es simple, un movimiento (cualquiera que sea) da 1 punto. Lo que implica que tener un mejor puntaje es tener menos puntos, pues si llegara a resolver el caso con un puntaje bajo significa que lo hizo en pocos movimientos.
12. Los espacios internos, al empezar el juego estarán marcados con el símbolo “o”, ello indica que la ubicación aún no se ha visitado. Al moverse a una ubicación es posible que se encuentren 2 cosas:
1. Espacio vacío “ ”, quiere decir que allí hay calle abierta, se debe hacer visible y el detective pasará a la siguiente ubicación que desee continuar.
2. Un callejón cerrado “[” que indica que allí no hay paso y será necesario que el usuario busque una nueva ruta para desplazarse.
13. En cualquier momento el usuario digitando la tecla “T” podrá ver la estructura de datos con las pistas que ha recogido.
14. El usuario podrá usar una Pista con la letra “X” (desde que tenga pistas recolectadas) para lograr beneficios, pero ello implica volver a encontrar esa pista en la ciudad, pues se reubicará aleatoriamente y el mapa se volverá a llenar con “o” para tapar los espacios ya descubiertos (los callejones cerrados pueden quedar expuestos, opcional). Los beneficios y condiciones al usar una pista serán los siguientes:

- a. Si la pista es una Huella, se reduce a la mitad los puntos que haya acumulado hasta el momento.
 - b. Si la pista es una Coartada, se eliminan 2 Callejones cerrados del tablero de forma aleatoria; ello podría eliminar callejones ya descubiertos u ocultos.
 - c. Si la pista es un Testimonio, aleatoriamente pueden pasar 2 cosas (se reduce a cero el puntaje, o se multiplica por 2 el puntaje).
 - d. Si la pista es una Prueba forense, el detective “D” podrá ir aleatoriamente a cualquier otra parte del tablero. Fíjese que la nueva ubicación no sea un lugar ya descubierto ni un callejón cerrado.
15. Los Sospechosos del caso deben mantenerse en una Tabla Hash, con un total de 8 sospechosos generados al inicio de la partida (puede tener una lista predefinida de nombres y atributos de los cuales escoger 8 al azar). Cada sospechoso tendrá nombre y un conjunto de atributos (por ejemplo: estatura, color de cabello, color de piel, forma de nariz, sexo, etc.). Uno de los 8 será marcado internamente y al azar como el culpable real del caso. La Tabla Hash debe permitir la búsqueda eficiente por nombre.
16. Cada vez que el detective recoja una Pista, el sistema deberá revelar al usuario un dato verídico sobre el culpable (un atributo). Esa información se debe acumular para que el usuario, al momento de la acusación, pueda discriminar entre los 6 sospechosos. En cualquier momento, digitando la tecla “S”, el usuario podrá ver la Tabla Hash de sospechosos con los atributos que ya se han revelado del culpable.
17. Algunas ubicaciones de la ciudad tendrán Testigos, marcados con la letra “W” (deberán aparecer 5 al azar en posiciones que no coincidan con pistas ni callejones). Cuando el detective pase por una ubicación con un Testigo, su declaración entra automáticamente a una Cola de declaraciones por procesar. En cualquier momento el usuario, digitando la tecla “I”, podrá interrogar al siguiente testigo de la cola, lo que dará como resultado revelar un atributo adicional del culpable (igual que las pistas, pero a través de la cola).
18. Cuando el detective haya recogido las 10 pistas, el juego pasará a la fase de acusación. El sistema mostrará la Tabla Hash de sospechosos con todos los atributos revelados hasta el momento, y le pedirá al usuario que escriba el nombre del sospechoso al que desea acusar. La búsqueda de ese nombre en la Hash debe ser explícita (mostrar al sustentador que la búsqueda es $O(1)$ promedio). Si la acusación es correcta, el detective gana el caso. Si es incorrecta, el caso se cierra como fracasado y el puntaje se penaliza al doble.
19. Ustedes deberán mantener un score histórico de los detectives que han jugado, con un ABB (Árbol Binario de Búsqueda).
20. En caso de almacenarse el puntaje de un detective que ya ha jugado antes, se debe conservar el detective con el mejor puntaje obtenido, es decir que, si ya está la detective “Mariana” con 32 puntos y ahora se desea guardar con 36 puntos, entonces se mantendrá el inicial que tenía 32, al ser un mejor puntaje.
21. El puntaje se guardará en la estructura de datos cuando el detective haya finalizado el caso (resuelto o fracasado).
22. Debe existir una funcionalidad en el programa que permita, dado un detective, observar si ha jugado antes y cuál ha sido su mejor puntaje.
23. Debe existir una funcionalidad que permita visualizar todos los detectives que tienen puntaje registrado con un orden, del menor puntaje al mayor; es decir, primero muestra al que tiene 25 que al que tiene 33.

A continuación, unas imágenes de ejemplo de momentos del juego:

Momento inicial del juego:

Mariana, Tu puntaje actual es: 0

o o o o o o o o o #
o o o o o o o o o #
o o o o o o o o o #
o o o o D o o o o #
o o o o o o o o o #
o o o o o o o o o #
o o o o o o o o o #
o o o o o o o o o #
o o o o o o o o o #
#

Luego de haber digitado T para ver las Pistas recogidas:

Mariana, Mira las pistas que llevas

```
[ '#      #' ]
[ '#      #' ]
[ '# T    #' ]   <- última (la que se usaría con X)
[ '# C    #' ]   <- penúltima
[ '# H    #' ]   <- antepenúltima
[ '# # #  #' ]
```

Luego de haber digitado S para ver la Tabla Hash de sospechosos:

Mariana, sospechosos del caso (atributos del culpable revelados hasta ahora):

Carlos		atributos confirmados:	zurdo, alto
Diana		atributos confirmados:	alta
Eduardo		atributos confirmados:	-
Fernanda		atributos confirmados:	alta, cabello rojo
Gonzalo		atributos confirmados:	-
Hilda		atributos confirmados:	alta

Tablero en una jugada intermedia:

Mariana, Tu puntaje actual es: 16

[illegible]

Jugada luego de usar una Huella con “X”:

Mariana, Tu puntaje actual es: 8

Y la Huella volvió al mapa

```
# # # # # # # # # #
# o o o | o o o o o #
# o o o o o o o o o #
# o o o o D | o o o #
# o o | o o o o o o #
# o o o o o o o o o #
# o o o o o o o o o #
# o o o o o o o o o #
# o o o o o o o o o #
# o o o o o o o o o #
# o o o o o o o o o #
# # # # # # # # # #
```

Fase de acusación (luego de recolectar las 10 pistas):

Mariana, has recolectado las 10 pistas. Es momento de acusar.

Sospechosos disponibles (Tabla Hash):

Carlos, Diana, Eduardo, Fernanda, Gonzalo, Hilda

Atributos confirmados del culpable: alta, cabello rojo

¿A quién acusas? > Fernanda

¡Caso resuelto! Fernanda era la culpable.

Puntaje final: 47 movimientos.

Rúbrica de propiedad intelectual para la sustentación

	1	0.8	0.6	0.4	0.2	0
Sustentación	Es evidente que el estudiante entiende el código que desarrolló, lo explica con claridad y responde correctamente a las preguntas.	Se logra evidenciar que el trabajo no fue hecho por la IA o por otra persona. La sustentación es buena, aunque hay inseguridad para explicar algunas partes del trabajo desarrollado o para responder algunas preguntas.	Se logra evidenciar que el trabajo no fue hecho por la IA o por otra persona. La sustentación es aceptable, se nota que el estudiante desarrolló el trabajo, pero le cuesta trabajo explicar aspectos del código.	La sustentación no es buena, hay inseguridad del estudiante para explicar gran parte del trabajo desarrollado o para responder muchas de las preguntas. Pero se logra evidenciar que el trabajo no fue hecho por la IA o por otra persona	El estudiante no logra responder de forma adecuada las preguntas dando clara evidencia de que todo o muchas partes del código fueron realizadas por otra persona o por la IA	Se evidencia que el estudiante no es el autor del código